

ALPS: Optimization-Based One-Shot Pruning for LLMs

Research Summary

1 Executive Summary & Research Context

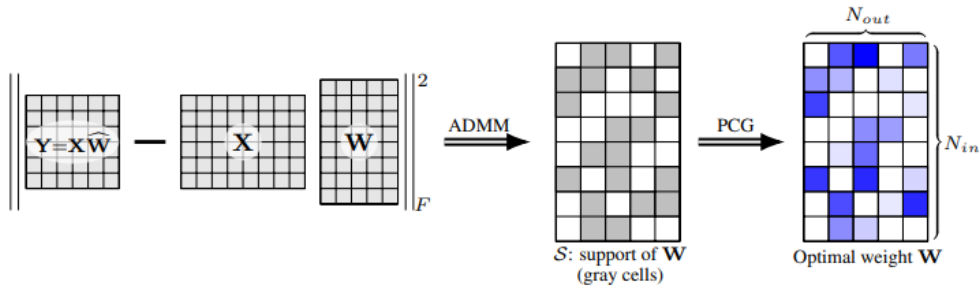


Figure 1: Overview of the proposed *ALPS* algorithm. **(Left)** The pruning problem with a layerwise reconstruction objective and an ℓ_0 constraint on the weights (Section 3.1). **(Middle)** ADMM with a ρ -update scheme (Algorithm 1) is employed to determine high-quality support for the weight matrix W (Section 3.2). **(Right)** The optimization problem is restricted to the obtained support, and a modified PCG method (Algorithm 2) is used to solve for the optimal weight values within the support (Section 3.3).

1.1 The Problem

Large Language Models (LLMs) like LLaMA-3 or OPT-175B have billions of parameters. Storing a model like OPT-175B in FP16 requires at least 320 GB of memory. This creates a bottleneck for widespread adoption, limiting deployment to organizations with access to massive computational clusters.

1.2 The Goal

We want to *prune* the model—setting specific weights to zero—so we can delete them. Ideally, we remove 70% to 90% of the weights while the model remains “smart” (low perplexity, high accuracy).

1.3 Constraint: One-Shot Pruning

We must do this in a “one-shot” manner: take the fully trained model, prune it once, and finish. We cannot afford iterative fine-tuning because retraining LLMs takes weeks of compute.

1.4 Historical Context

Previous pruning research includes **Optimal Brain Damage (OBD)** and **Optimal Brain Surgeon (OBS)**, which used second-order Hessian information to determine importance. The **Lottery Ticket Hypothesis** further spurred interest by suggesting dense networks contain sparse, trainable subnetworks. While these methods were effective for smaller networks, they are often computationally prohibitive for LLMs.

1.5 The Gap in Current Methods

Current state-of-the-art methods (SparseGPT, Wanda, DSnoT) rely on heuristics. For example, “delete the smallest weight.” While effective at low sparsity ($< 50\%$), these approximations fail at high sparsity because they do not solve the true optimization problem, leading to significant performance degradation.

1.6 The ALPS Solution

ALPS (ALternating direction method of multipliers for Pruning at Scale) proposes a rigorous mathematical framework. Instead of guessing, it treats pruning as a constrained optimization problem. It uses ADMM to decide which weights to keep and PCG to calculate the exact new values, proving that rigorous optimization is feasible at LLM scale.

2 Glossary of Key Concepts

Weights (W)

The adjustable parameters of the model.

Pruning

Setting neural network weights to zero (removing connections).

Unstructured Pruning

Any weight anywhere can be removed. Flexible but harder to accelerate on hardware.

N:M Sparsity

Hardware-friendly structure (e.g., NVIDIA 2:4) where for every block of M weights, only N can be non-zero.

Input Activations (X)

The data flowing into the layer.

Reconstruction Error

The difference between the output of the original layer ($X\hat{W}$) and the pruned layer (XW).

Perplexity (PPL)

A metric measuring how “confused” the model is. Lower is better.

ℓ_0 Norm

A mathematical way to count non-zero elements.

Hessian Matrix (H)

A matrix of second derivatives ($X^T X$) describing the curvature of the loss function.

Heuristic vs. Optimization

Heuristics are fast “rules of thumb” (approximate). Optimization is “solving the equation” (rigorous).

3 Mathematical Formulation

ALPS operates on a layer-wise basis. The core philosophy is to minimize the difference between the original layer’s output and the pruned layer’s output.

3.1 The Objective Function

$$\min_W \underbrace{\|X\hat{W} - XW\|_F^2}_{\text{Reconstruction Error}} + \underbrace{\frac{\lambda}{2}\|\hat{W} - W\|_F^2}_{\text{Ridge Regularization}} \quad \text{s.t.} \quad \|W\|_0 \leq k \quad (1)$$

3.2 Detailed Breakdown

- **X (Input Activations):** Data flowing into the layer, derived from calibration data.
- **$\hat{W}$ (Dense Weights):** The original, frozen weights of the model (also referred to as W_c in algorithms).
- **W (Sparse Weights):** The new weights we are trying to find.
- **Reconstruction Term:** Ensures the pruned layer mimics the original using Frobenius norm.
- **Ridge Regularization ($\frac{\lambda}{2}$):** Critical safety mechanism preventing W from drifting too far from $\hat{W}$.
- **ℓ_0 Constraint ($\|W\|_0 \leq k$):** Enforces the sparsity budget (keep only k non-zeros).
- **Hessian Definition (H):** We define $H = X^T X + \lambda^2 I$ to simplify the optimization steps.

4 The ALPS Methodology

Since the problem is hard to solve directly, ALPS splits it into two stages: finding the support (ADMM) and refining the values (PCG).

4.1 Part 1: Finding the Support via ADMM

ALPS uses *Operator Splitting* (Alternating Direction Method of Multipliers) to split the problem into continuous optimization and discrete selection. To apply ADMM, we introduce an auxiliary variable D and the Augmented Lagrangian function L_ρ :

$$L_\rho(W, D, V) = \|X\hat{W} - XW\|_F^2 + \frac{\lambda^2}{2}\|\hat{W} - W\|_F^2 + \langle V, W - D \rangle + \frac{\rho}{2}\|W - D\|_F^2 \quad (2)$$

subject to the sparsity constraint on D ($\|D\|_0 \leq k$).

Algorithm 1 ADMM for layer-wise pruning with ℓ_0 constraint

- 1: **Input:** Initial penalty ρ_0 .
 - 2: Initialize $V^{(0)} = 0_{N_{in} \times N_{out}}$ and $D^{(0)} = W^{(0)} = W_c$
 - 3: **for** $t = 0, 1, \dots$ **do**
 - 4: Update $W^{(t+1)}$, $D^{(t+1)}$ and $V^{(t+1)}$ according to Eq. (4) with $\rho = \rho_t$.
 - 5: Increase ρ_t to get ρ_{t+1} based on the change in the support of $\text{Supp}(D^{(t)})$.
 - 6: **end for**
-

4.1.1 The Iterative Updates

W-Update (Minimizing Error):

$$W^{(t+1)} = (H + \rho I)^{-1}(H\hat{W} - V^{(t)} + \rho D^{(t)}) \quad (3)$$

Computational Efficiency: The matrix H is constant for a given layer. ALPS pre-computes the Eigenvalue decomposition $H = Q\Lambda Q^T$. This allows the inverse $(H + \rho I)^{-1}$ to be updated efficiently as ρ changes without a full matrix inversion each time.

D-Update (Enforcing Sparsity):

$$D^{(t+1)} = P_k \left(W^{(t+1)} + \frac{V^{(t)}}{\rho} \right) \quad (4)$$

V-Update (Dual Ascent):

$$V^{(t+1)} = V^{(t)} + \rho(W^{(t+1)} - D^{(t+1)}) \quad (5)$$

4.2 Part 2: The ρ -Update Scheme

The parameter ρ controls the penalty for $W \neq D$. ALPS starts with small ρ to explore and find the best support, then increases it to “freeze” the support.

4.3 Part 3: Weight Refinement via Vectorized PCG

Once the support stabilizes, ADMM converges slowly because it is solving a fixed-support quadratic problem. ALPS switches to a restricted solver for final polish.

4.3.1 The Solution: Preconditioned Conjugate Gradient (PCG)

ALPS employs a post-processing technique that fixes the support S of the current solution and refines the solution within this support. Crucially, this method achieves a **20x–200x speedup** compared to a naive backsolve approach (e.g., `torch.linalg.solve`).

Algorithm 2 PCG with vectorization for solving problem (6)

- 1: **Input:** Support S , pre-conditioner $M = \text{Diag}(H)$, initial solution W_0
 - 2: Set $R_0 := H(W_c - W_0)$
 - 3: Project R_0 onto the support S by setting all elements outside the support to zero.
 - 4: Set $Z_0 = M^{-1}R_0$ and $P_0 = Z_0$
 - 5: **for** $t = 0, 1, \dots$ **do**
 - 6: $\alpha_t = \text{Tr}(R_t^\top Z_t) / \text{Tr}(P_t^\top H P_t)$
 - 7: $W_{t+1} = W_t + \alpha_t P_t$
 - 8: $R_{t+1} = R_t - \alpha_t H P_t$
 - 9: Project R_{t+1} onto the support S by setting all elements outside the support to zero.
 - 10: $Z_{t+1} = M^{-1}R_{t+1}$
 - 11: **if** R_{t+1} is sufficiently small **then**
 - 12: **break**
 - 13: **end if**
 - 14: $\beta_t = \text{Tr}(R_{t+1}^\top Z_{t+1}) / \text{Tr}(R_t^\top Z_t)$
 - 15: $P_{t+1} := Z_{t+1} + \beta_t P_t$
 - 16: **end for**
-

5 Hardware Adaptation: N:M Structured Sparsity

ALPS adapts by modifying only the D-update step (the projection step) in the ADMM loop. In ADMM, the W-update “sees” the penalty for having zeros in the wrong places (via dual variable V). This allows ALPS to handle structured sparsity patterns like 2:4 efficiently.

6 Experimental Results & Technical Analysis

ALPS was tested against Magnitude Pruning, SparseGPT, Wanda, and DSnoT.

6.1 Experimental Setup

- **Models:** OPT (1.3B to 30B), LLaMA-2 (7B, 13B), and LLaMA-3 (8B).
- **Calibration Data:** 128 segments of 2048 tokens each, randomly selected from the first shard of the C4 dataset.
- **Metrics:** Perplexity (WikiText2, PTB, C4) and Zero-shot accuracy (MMLU, PIQA, LAMBADA, ARC-Easy, ARC-Challenge).

6.2 Ablation Studies

- **Support Quality:** When fixing the support and solving for optimal weights, the support found by ALPS yields **20% to 40% lower reconstruction error** compared to baselines like SparseGPT and Wanda.
- **PCG Efficiency:** Compared to the optimal "Backsolve" method, ALPS's PCG refinement achieves comparable reconstruction error but is **20x to 200x faster**.

Metric	Observation
Reconstruction	ALPS achieves lower layer-wise error (7.6% vs 12% on OPT-13B).
Perplexity	At 70% sparsity, ALPS reduces perplexity by 29% compared to baselines on LLaMA-3 8B.
N:M Sparsity	Magnitude Pruning fails (PPL ~ 1981), ALPS succeeds (PPL ~ 10.9).

Table 1: Summary of Experimental Results

7 Strengths, Limitations, and Practicality

- **Strengths:** Principled optimization, provable convergence, GPU-efficient PCG (20x-200x speedup), adaptability to structured sparsity.
- **Limitations:** Higher runtime and memory overhead than heuristics, though mitigated by the Eigenvalue decomposition trick and vectorized PCG.

8 Theoretical Guarantee

The ALPS algorithm provides rigorous convergence guarantees compared to heuristic methods.

Theorem 1. Let $\{D^{(t)}\}_{t=0}^{\infty}$ and $\{W^{(t)}\}_{t=0}^{\infty}$ be the sequences generated in Algorithm 1. Suppose the penalty parameter $\{\rho_t\}_{t=1}^{\infty}$ chosen in Algorithm 1 satisfies $\sum_{t=1}^{\infty} 1/\rho_t < \infty$. It then holds

$$\max \left\{ \|D^{(t+1)} - D^{(t)}\|_F, \|W^{(t+1)} - W^{(t)}\|_F \right\} \leq \frac{C}{\rho_t}, \quad (6)$$

where C is a constant depending on X , W_c , λ_2 , and $\sum_{t=1}^{\infty} 1/\rho_t$. In particular, there exists a matrix $\bar{D}$ such that $D^{(t)} \rightarrow \bar{D}$ and $W^{(t)} \rightarrow \bar{D}$ as $t \rightarrow \infty$.

This is a novel contribution because most ADMM results assume convexity/continuity, whereas ALPS handles non-convex ℓ_0 constraints.

9 Conclusion & Future Directions

The conclusion summarizes the two-stage philosophy of ALPS: **Decoupling** the combinatorial search (ADMM) from the continuous optimization (PCG).

9.1 Future Directions

- **Quantization Integration:** Combining pruning with quantization techniques (e.g., SpQR) to achieve joint compression.
- **Pruning-Aware Training:** Applying ALPS principles during the training process itself to find inherently sparse architectures.
- **Structured Extensions:** Further adaptation to block sparsity or pruning entire attention heads/transformer layers.